

Documentación Técnica: Pipeline de Datos Medallion en AWS

Proyecto: Steam Data — Hidden Gem Score (HGS)

Equipo: Equipo 3 — Curso Ingeniería de Datos — Facultad de Ingeniería, UNAM

Documentación técnica versión 1.0 — Mayo 2026

1. Configuración del Entorno

Esta sección detalla los componentes técnicos y las configuraciones específicas utilizadas para el procesamiento de datos del proyecto **Steam Data — Hidden Gem Score**. El pipeline opera sobre infraestructura AWS en la región us-east-2 e implementa una arquitectura Medallion (Bronze, Silver, Gold) para identificar videojuegos independientes de alta calidad con baja visibilidad en la plataforma Steam.

1.1 Dependencias Python

Las siguientes librerías deben estar instaladas en el entorno virtual de la instancia EC2:

```
pip install boto3 pandas psycpg2-binary pyarrow requests
```

Rol de cada dependencia: boto3 (interacción con S3), pandas (transformaciones y cálculo del HGS), psycpg2-binary (carga en RDS PostgreSQL), pyarrow (lectura/escritura de Parquet), requests(consumo de la Steam API en la capa Bronze).

1.2 Activar entorno virtual y cargar variables de entorno

```
source /home/ubuntu/venv/bin/activate
export S3_BUCKET="unam-2026-ingenieriadedatos-equipo3-660864588540-us-east-2-an"
export
DB_HOST="unamdatabaseingdedatoequipo3.cpswcmwoolha.us-east-2.rds.amazonaws.com"
export DB_NAME="steam_gold"
export DB_USER="dbadmin"
export DB_PASSWORD="<contraseña>"
export S3_PREFIX="2silver/" # prefijo Silver; Bronze usa 1bronze/, Gold usa 3gold/
```

2. Inventario de Recursos de Infraestructura

Servicio AWS	Nombre / Identificador	Función Principal
Amazon S3	unam-2026-ingenieriadedatos-equip o3-660864588540-us-east-2-an	Data Lake — almacena las tres capas (1bronze/, 2silver/, 3gold/). Región us-east-2.
AWS Glue	TEST-GLUE-JOB-DATA-QUALITY / IAM Role: TEST-GLUE-ROLE	ETL Bronze→Silver con 12 reglas EvaluateDataQuality. Región us-east-2.
Amazon EC2	ubuntu@ip-172-31-47-107 (Ubuntu 18.3) — us-east-2	Ingesta Bronze (CRON) y ejecución de gold_layer.py.
Amazon EventBridge	EventBridge Scheduler — orquestación del CRON de ingesta	Automatización de la ingesta periódica desde la Steam API pública.
Amazon RDS	unamdatabaseingdedatoequipo3.cp swcmwoolha.us-east-2.rds.amazona ws.com	PostgreSQL 18.3 — Data Mart Gold (base: steam_gold, puerto 5432, SSL TLSv1.3 AES-256-GCM).
Streamlit Dashboard	Capa de consumo — conecta al RDS Gold	Visualización interactiva del HGS y rankings de joyas ocultas.

3. Especificaciones de los Pipelines (ETL)

3.1. Ingesta — Capa Bronze (Steam API → S3)

Un script Python en EC2 consume la **Steam API pública (gratuita)** y escribe cada run en S3 bajo 1bronze/ en formato JSONL (una línea JSON por juego). El scheduler es un CRON orquestado por EventBridge Scheduler.

Campos extraídos por juego desde la Steam API: appid, nombre, géneros, precio, puntuación global, total de reseñas, reseñas positivas, reseñas negativas, jugadores actuales (API), Metacritic score. Estos son los campos que alimentan tanto las reglas DQ de la capa Silver como el cálculo del HGS en Gold.

3.2. Transformación — Capa Silver (AWS Glue ETL + Data Quality)

El job **TEST-GLUE-JOB-DATA-QUALITY** ejecuta 5 pasos en secuencia: (1) Data source S3 → (2) Change Schema → (3) Evaluate Data Quality → (4) SelectFromCollection (originalData) → (5) Data target S3. Solo los registros que pasan las 12 reglas llegan a la capa Silver en formato Parquet.

Las 12 reglas de calidad de datos (EvaluateDataQuality) se agrupan en tres categorías:

- **Complejidad (5 reglas):** nombre y appid sin nulos (sin estos no hay score ni identificación); total_resenas, resenas_positivas y resenas_negativas sin nulos (sin ellos el 50% del HGS es incalculable); puntuacion sin nulos (métrica principal de calidad percibida).
- **Rangos válidos (4 reglas):** metacritic_score entre 0 y 100; puntuacion entre 0 y 100; jugadores_actuales_api ≥ 0 ; resenas_positivas y resenas_negativas ≥ 0 .
- **Consistencia lógica (3 reglas):** total_resenas ≥ 10 (menos de 10 no es representativo para el score); resenas_positivas + resenas_negativas $\leq$ total_resenas (la suma de partes no puede superar el total); appid único por registro (sin duplicados en la capa Silver).

3.3. Refinamiento — Capa Gold (gold_layer.py → RDS + S3)

Lee los Parquet validados de Silver, aplica drop_duplicates por appid, calcula el HGS (los scores intermedios se calculan en memoria y no se persisten en el RDS), clasifica en tiers y carga en RDS con INSERT ... ON CONFLICT (appid) DO UPDATE para idempotencia.

Finalmente persiste el resultado en 3gold/hidden_gems_final.parquet para análisis histórico con Athena. Resultado en producción: 10,538 juegos, 707 géneros.

Fórmula del Hidden Gem Score (calculada en memoria, no almacenada en RDS):

quality_score = resenas_positivas / total_resenas # peso 50%

obscurity_score = 1 - (total_resenas / max_resenas_del_dataset) # peso 30%

price_score = 1.0 si precio==0 else max(0.0, 1-(precio/60)) # peso 20%

hidden_gem_score = (0.50*quality + 0.30*obscurity + 0.20*price).round(4)

tier = "S" si hgs ≥ 0.85 | "A" si hgs ≥ 0.70 | "B" si hgs < 0.70

Nota: los juegos Free-to-Play (precio = 0.0) reciben price_score = 1.0 automáticamente, beneficiando su accesibilidad en el índice.

4. Diccionario de Datos (Capa Gold)

La base de datos steam_gold contiene dos tablas confirmadas en producción (verificadas con \dten psq).

DDL — Tabla hidden_gems (esquema confirmado en producción):

CREATE TABLE public.hidden_gems (

```

appid          BIGINT          NOT NULL,
nombre         TEXT,
generos        TEXT,
precio         NUMERIC(8,2),
puntuacion     NUMERIC(5,4),
total_resenas  INTEGER,
resenas_positivas INTEGER,
hidden_gem_score NUMERIC(5,4),
tier           CHARACTER(1),
updated_at     TIMESTAMP WITHOUT TIME ZONE DEFAULT now(),
CONSTRAINT hidden_gems_pkey PRIMARY KEY (appid)
);

```

Diccionario de campos — hidden_gems:

Campo	Tipo	Descripción
appid	BIGINT (PK)	Identificador único del juego en Steam. Llave primaria (índice btree).
nombre	TEXT	Título oficial del videojuego en Steam. Sin nulos (regla DQ de completitud).
generos	TEXT	Géneros del juego (707 valores distintos en producción). Fuente de la tabla genre_summary.
precio	NUMERIC(8,2)	Precio en USD. Valor 0.00 indica Free-to-Play (price_score = 1.0 automáticamente).
puntuacion	NUMERIC(5,4)	Puntuación global (0–100). Métrica principal de calidad percibida. Sin nulos (regla DQ).
total_resenas	INTEGER	Total de reseñas. Debe ser ≥ 10 (regla DQ). Insumo del obscurity_score (30% del HGS).

Campo	Tipo	Descripción
resenas_positivas	INTEGER	Insumo principal del quality_score (50% del HGS). ≥ 0 (regla DQ de rango).
hidden_gem_score	NUMERIC(5,4)	Índice ponderado final (0.0000–1.0000). Métrica central. Los scores intermedios (quality, obscurity, price) se calculan en memoria y no se almacenan.
tier	CHAR(1)	S (HGS ≥ 0.85 , alta prioridad para promoción) A (0.70–0.84) B (< 0.70, en observación).
updated_at	TIMESTAMP	Fecha y hora de la última actualización del registro. Default: now(). Permite trazabilidad temporal entre runs.

Tabla derivada: **genre_summary**

Agrega estadísticas por género para responder: "¿Qué géneros concentran más joyas ocultas sin explotar?". Campos: género, conteo de juegos, HGS promedio (avg_hgs), y distribución de tiers por género. Se consulta con: `SELECT * FROM genre_summary ORDER BY avg_hgs DESC LIMIT 5;`

Vista de negocio: **top_hidden_gems**

Expone directamente los juegos Tier S (HGS ≥ 0.85) ordenados por score, abstrayendo la complejidad de la tabla principal. Si la lógica de clasificación cambia, solo se actualiza esta vista; el dashboard Streamlit no requiere reprogramación.

5. Guía de Ejecución y Monitoreo

Orden de ejecución completo del pipeline:

- **Bronze (automático):** El CRON de EC2, orquestado por EventBridge, extrae de la Steam API y deposita en S3 (1bronze/). No requiere intervención manual.
- **Silver (manual o programado vía CLI):** `aws glue start-job-run --job-name`

TEST-GLUE-JOB-DATA-QUALITY

- **Gold (manual en EC2):** *python gold_layer.py* (con el venv activo y las variables de entorno cargadas).

Salida esperada en consola al ejecutar *gold_layer.py* (confirmada en producción):

- Leyendo Silver Layer desde S3... 10538 juegos cargados.
- Calculando Hidden Gem Score... 707 géneros procesados.
- Cargando en RDS PostgreSQL... ► Limpiando duplicados... ✓ Datos cargados correctamente.

✓ Gold Layer completada.

Validación post-ejecución (conectarse al RDS con psql):

```
psql "host=$DB_HOST port=5432 dbname=steam_gold user=dbadmin sslmode=verify-full  
sslrootcert=./global-bundle.pem"
```

```
\dt                                -- Debe listar: genre_summary, hidden_gems  
SELECT COUNT(*) FROM hidden_gems;    -- Debe retornar 10538  
SELECT tier, COUNT(*) FROM hidden_gems GROUP BY tier;    -- Distribución S/A/B  
SELECT * FROM genre_summary ORDER BY avg_hgs DESC LIMIT 5; -- Top géneros  
SELECT * FROM top_hidden_gems;        -- Joyas ocultas Tier S
```

Troubleshooting básico:

- **gold_layer.py falla con error de conexión RDS:** Verificar que las variables de entorno *DB_HOST*, *DB_USER* y *DB_PASSWORD* estén cargadas y que el *Security Group* del RDS permita tráfico desde la IP de la instancia EC2.
- **read_silver retorna 0 frames:** Confirmar que el Glue Job completó exitosamente y que existen archivos *.parquet* en S3 bajo el prefijo *2silver/*. Revisar *CloudWatch Logs* del job.
- **Glue Job falla en EvaluateDataQuality:** Revisar en *Amazon CloudWatch Logs* el detalle de qué regla falló. Las causas más comunes son registros con *total_resenas < 10* o nulos en campos críticos provenientes de la *Bronze*.